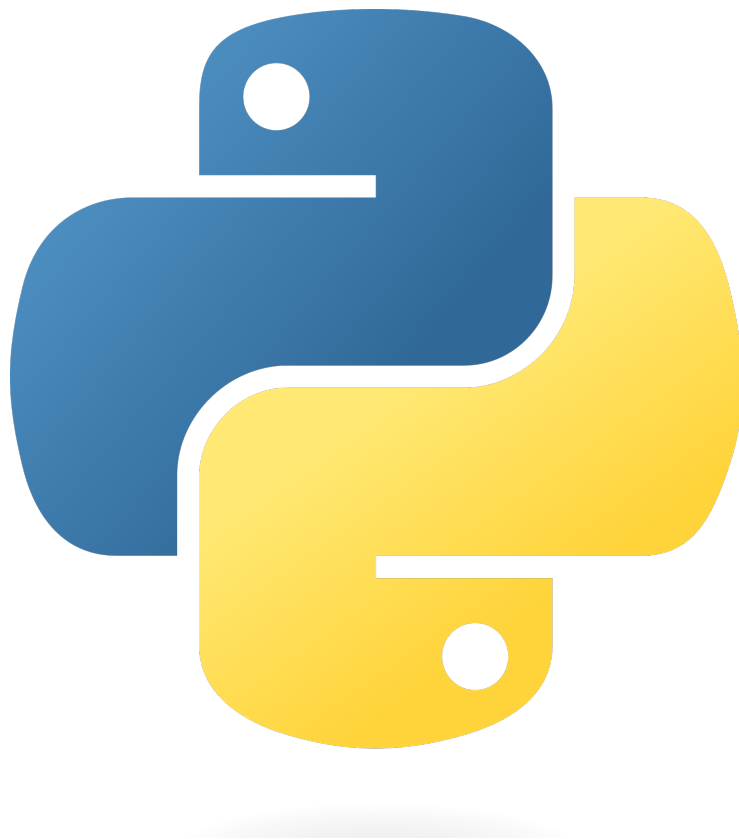


# Sending Super Secret Messages with a Book Cipher

Python Programming Project



Gaël KERNINON and Clément GOMEZ

Cracow University of Technology, Poland 

May 15, 2026

## Contents

|          |                                                                         |           |
|----------|-------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                     | <b>3</b>  |
| <b>2</b> | <b>History of the Book Cipher</b>                                       | <b>4</b>  |
| 2.1      | History of the <i>Rebecca</i> Cipher . . . . .                          | 4         |
| <b>3</b> | <b>Methodology</b>                                                      | <b>5</b>  |
| 3.1      | Encoding Process . . . . .                                              | 5         |
| 3.2      | The Switch Mechanism . . . . .                                          | 5         |
| 3.3      | Handling Missing Words . . . . .                                        | 5         |
| <b>4</b> | <b>Implementation</b>                                                   | <b>6</b>  |
| 4.1      | Project Architecture . . . . .                                          | 6         |
| 4.2      | Data Acquisition and Processing ( <code>read_file.py</code> ) . . . . . | 6         |
| 4.3      | Encoding ( <code>encode.py</code> ) . . . . .                           | 7         |
| 4.4      | Decoding ( <code>decode.py</code> ) . . . . .                           | 8         |
| <b>5</b> | <b>Execution and Results</b>                                            | <b>9</b>  |
| 5.1      | Example Execution . . . . .                                             | 9         |
| <b>6</b> | <b>Security Analysis, Limitations, and Improvements</b>                 | <b>10</b> |
| 6.1      | Lack of Code Obfuscation . . . . .                                      | 10        |
| 6.2      | Pseudorandom Number Generation (PRNG) . . . . .                         | 10        |
| 6.3      | Vulnerability of the Switch . . . . .                                   | 10        |
| <b>7</b> | <b>Conclusion</b>                                                       | <b>11</b> |

# 1 Introduction

This document serves as the project report for the Secret Message Project, developed during the 2026 Erasmus+ semester at the *Cracow University of Technology* in Poland.

The main objective of this student project was to build a functional cryptographic tool utilizing Python programming techniques learned in the course.

The core of the project is the implementation of a computational Book Cipher, an ancient cryptographic technique modernized through programmatic file processing, array manipulation, and pseudo-random index generation.

We sought to understand how text manipulation and data structures like dictionaries and lists can be effectively used to encode and decode information safely.

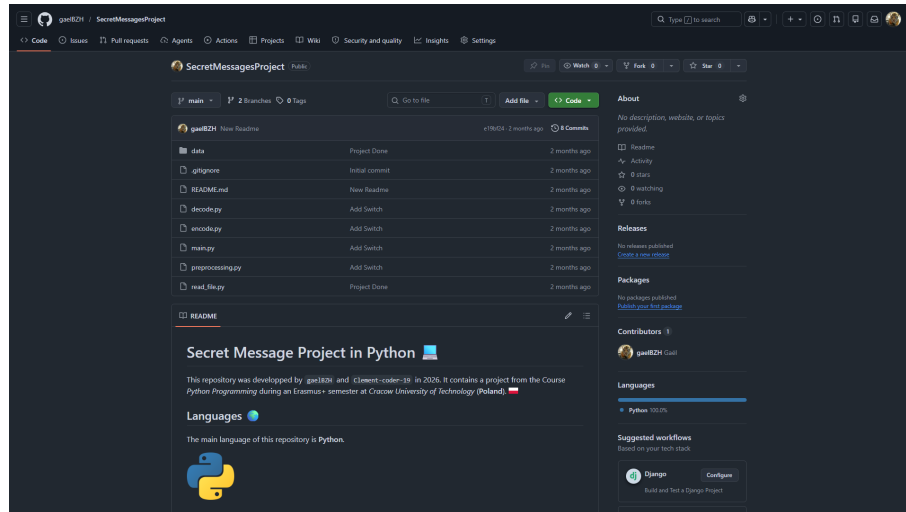


Figure 1: GitHub Repository of the Project

## 2 History of the Book Cipher

The book cipher is a well known cipher in cryptographic history. Instead of using a mathematical algorithm to scramble letters, a book cipher uses a large piece of text, typically a book as the key. Both the sender and the receiver must possess exact copies of the text. Words or letters in the secret message are replaced by their corresponding location<sup>1</sup> in the key text.

### 2.1 History of the *Rebecca* Cipher

One of the most famous utilizations of a book cipher occurred during **World War II**.

German spies stationed in Allied territories would frequently use widely circulated novels to encode their communications. According to historical records and espionage lore, Daphne du Maurier's novel *Rebecca* was a prominent choice for this purpose.

The availability of the book made it an ideal key, as carrying it would not immediately arouse suspicion compared to carrying a standard codebook.

In our original project design, we intended to use *Rebecca* as the source text (`data/lost.txt`) to accurately reflect this rich historical context. However, we encountered copyright limitations. To ensure strict compliance with modern public domain and intellectual property laws, we substituted *Rebecca* with Sir Arthur Conan Doyle's *The Lost World*. Although the historical context is slightly altered, the mathematical and computational principles of the cipher remain completely identical.

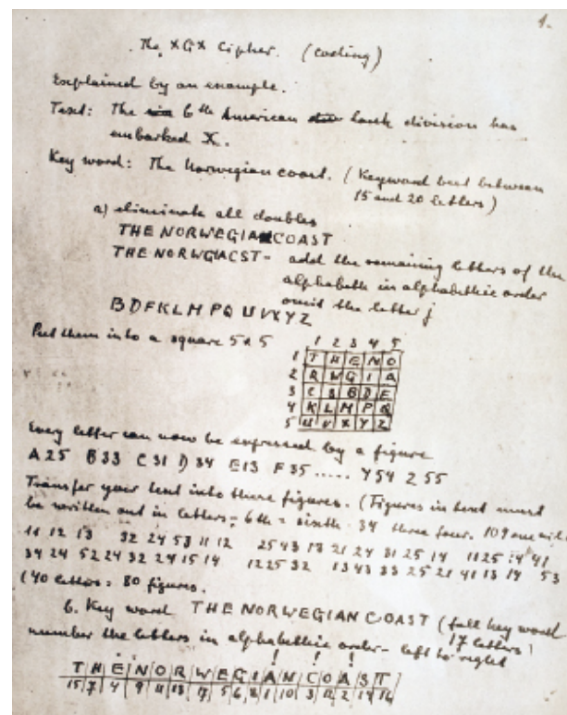


Figure 2: Transposition Manual for the Rebecca Code. (National Library of Ireland)

<sup>1</sup>e.g., page, line, and word number

### 3 Methodology

Our methodology for digitalizing the book cipher revolves around parsing the book into a list of constituent, punctuation-free words and building lookup dictionaries (histograms) for encoding and decoding.

#### 3.1 Encoding Process

Instead of page and line numbers, our Python implementation uses the absolute index of the word in the cleaned text corpus. Because standard words like *the*, *and*, or *is* appear thousands of times, the program randomly selects one of the many available indexes. This introduces a form of substitution, ensuring that the same plaintext word does not encode to the same ciphertext integer every time. This stops simple frequency analysis.

#### 3.2 The Switch Mechanism

To add an extra layer of security, we implemented a switch variable. This integer offset is added to the word index during encoding and subtracted during decoding. This acts as a secondary key. Even if an adversary discovers that *The Lost World*<sup>2</sup> is the source text, they would still need the correct switch value to decrypt the array of integers properly.

#### 3.3 Handling Missing Words

A common flaw in book ciphers is the inability to encode proper nouns, modern slang, or specific terms that do not appear in the source text. We resolved this through a letter-by-letter fallback system. When a word is not found in the book, the algorithm switches to "Letter Mode". It signals the beginning of an unknown word by encoding the word "the" twice consecutively. Following this, it spells the unknown word out by picking the indexes of random words that begin with the required letters. Finally, it signals the end of the spelled word by encoding the word "a" twice consecutively.

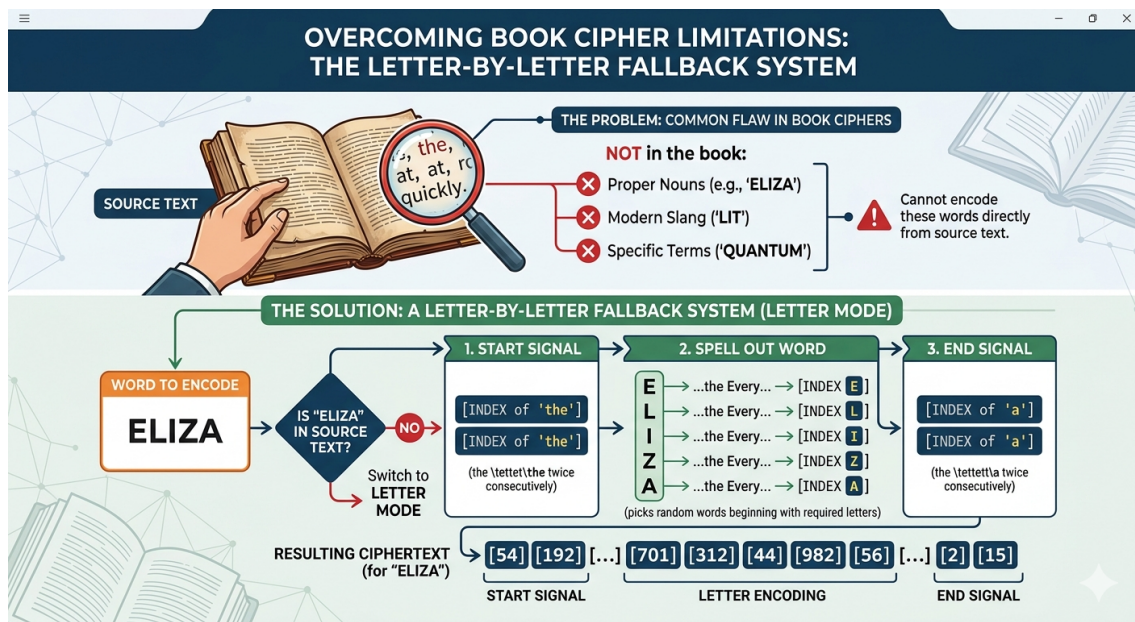


Figure 3: Flowchart and Example of Encoding Missing Word ELIZA. (AI-Generated Image)

<sup>2</sup>Selected book as a replacement for Rebecca for copyright reasons. See 2.1

## 4 Implementation

The project is structured into several distinct Python modules to maintain code readability and adhere to software engineering best practices. The source code is available on GitHub :

<https://github.com/gaelBZH/SecretMessagesProject>.

### 4.1 Project Architecture

The repository structure is as follows:

```
1 SecretMessagesProject
2 |--- data
3 |--- |-- lost.txt
4 |--- decode.py
5 |--- encode.py
6 |--- main.py
7 |--- preprocessing.py
8 |--- read_file.py
9 |--- README.md
```

The folder `data/` contains the source file `lost.txt` (*Lost of the World*), used to encode and decode messages.

The modules `encode.py` and `decode.py` contain the core functions for message processing.

The modules `preprocessing.py` and `read_file.py` provide helper functions to improve program readability and efficiency.

The script `main.py` is the entry point of the project.

### 4.2 Data Acquisition and Processing (`read_file.py`)

The first step is reading the book. We defined `read_file(filename)` to remove punctuation and returning a list of alphabetic words. Following this, we defined `create_histograms(words)` to create two dictionaries: `D_words` and `D_letters`.

```
1 def create_histograms(words):
2     D_words = {}
3     D_letters = {}
4     for i in range(len(words)):
5         word = words[i].lower()
6         letter = word[0]
7
8         # First Dictionnary
9         if word in D_words:
10             D_words[word].append(i)
11         else:
12             D_words[word] = [i]
13
14         # Second Dictionnary
15         if letter in D_letters:
16             D_letters[letter].append(i)
17         else:
18             D_letters[letter] = [i]
19     return D_words, D_letters
```

This dual-dictionary approach optimizes the time complexity of the encoding phase from  $O(n^2)$  down to  $O(n)$ , utilizing space complexity in exchange for execution speed.

### 4.3 Encoding (encode.py)

The encoding module handles the logic of matching text to book indexes. It relies heavily on the `random.randint()` function to select an index from the dictionary

```

1 def get_index_word(word, D_words):
2     if (word_exists(word, D_words)):
3         # Get all indexes of word in the dictionary
4         indexes = D_words[word]
5
6         # Returns one random index of word
7         return indexes[randint(0, len(indexes)-1)]
8
9     else:
10        return -1 # word does not exist in the dictionary

```

When a word returns -1, the script invokes `encode_proper_word()` function, which injects the "the the" and "a a" flags, passing through get index letter for character-by-character mapping.

#### Implementation Limits

As stated previously, when encoding proper nouns, we need to iterate through each letter of the word and find an index that matches that letter in the `D_letters` dictionary. Let's see how many indices are available for every possible letter in the alphabet:

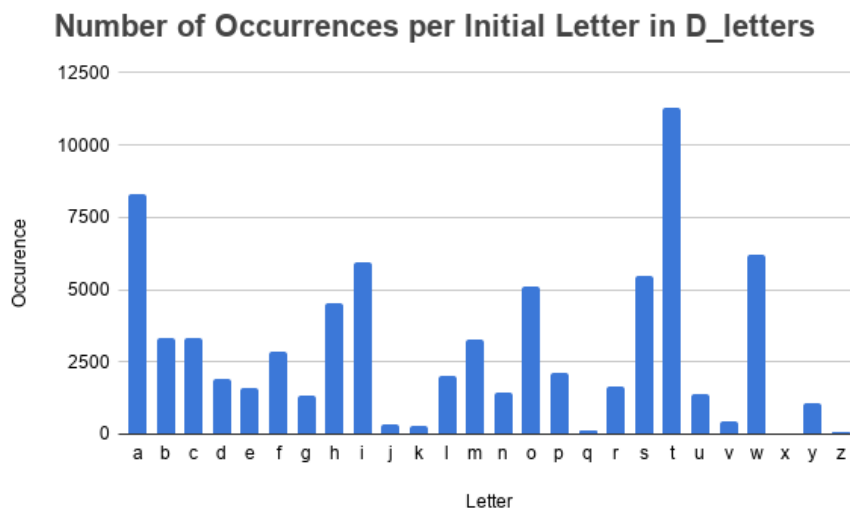


Figure 4: Analysis of Letter Frequencies in *The Lost World* source book.

As you might notice, during the construction of the `D_letters` dictionary, it was observed that the letter 'x' did not appear as an initial character in any word within the selected corpus of *The Lost World*. This absence posed a critical limitation to the 'Letter Mode' fallback system.

Without at least one valid index for every letter of the alphabet, it would be mathematically impossible to encode proper nouns or unknown terms containing the letter 'x' !

To resolve this structural issue and ensure the robustness of the cryptographic tool, a single word 'xray' was manually appended to the end of the source text. This modification created the necessary occurrence required to populate the dictionary.

## 4.4 Decoding (decode.py)

The decoding process is the inverse operation, designed to reconstruct the original message from the array of shifted integers. It subtracts the "switch" value from each integer to find the original word's position in the book's word list.

### Decode Word

At its core, individual units of the message are resolved by the `decode_word()` function, which performs the index-to-string mapping.

```
1 def decode_word(i, data, switch):  
2     return data[i - switch]
```

### Decode the Entire Message

The `decode_text()` function implements a state machine to handle the transition between standard word decoding and character-by-character words using the "*Letter Mode*".

- In the default state, each index is resolved to its corresponding word in the book.
- Detecting *Letter Mode*: If the sequence "the the" is encountered, the decoder switches its internal state to start collecting words for character extraction.
- Reconstructing Proper Words: While in *Letter Mode*, the decoder accumulates words until detecting the "a a" termination sequence, it exits the mode and processes the accumulated list using `list_to_word()` from `preprocessing.py` to extract the first letter of each word and form a single word.

```
1 def decode_text(L, data, switch=0):  
2     result = []  
3     temp = []  
4     letter_mode = False  
5     for index in L:  
6         if letter_mode:  
7             temp.append(decode_word(index, data, switch))  
8         else:  
9             result.append(decode_word(index, data, switch))  
10  
11     # Logic for 'the the' and 'a a' triggers state changes  
12     if (len(result) >= 2 and\  
13         result[-1] == "the" and\  
14         result[-2] == "the"):  
15  
16         result.pop()  
17         result.pop()  
18         letter_mode = True  
19         temp = []  
20     if (len(temp) >= 2 and temp[-1] == "a" and temp[-2] == "a"):  
21         letter_mode = False  
22         temp.pop()  
23         temp.pop()  
24         txt = list_to_word(temp)  
25         if (txt != ""):  
26             result.append(txt)  
27     return result
```



## 5 Execution and Results

The entry point of the project is `main.py`. The script reads the source book `data/lost.txt`, prompts the user for a secret message and a switch value, and outputs the integer array. Immediately after, it runs the decode function to verify the integrity of the message.

### 5.1 Example Execution

As documented in our repository<sup>3</sup>, a standard run behaves as follows :

```
1 What is your message ? hello world
2 What is the switch ? 42
3 The encoded message is : [65408 , 32016 , 51046 , 20421 , 67481 ,
   15623 , 31938 , 46866 , 67107 , 39079]
4 The decoded message is : hello world
```

NB: Because we employ a random integer selection among duplicate words, encoding the exact same message "hello world" with the same switch of 42 may return an entirely different array of numbers on the next execution.

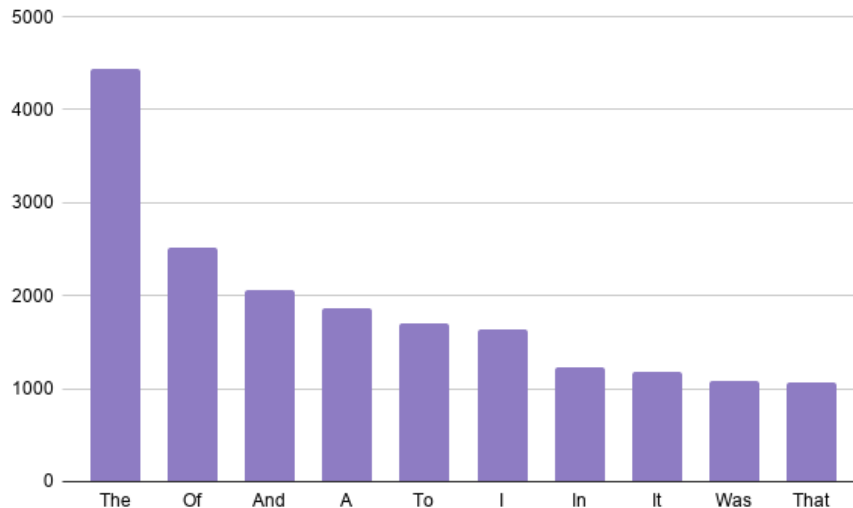


Figure 5: Frequency Distribution of Top 10 Words in `D_words`

While these ten words are the most common, they are part of a system supporting over thousands possible combinations. By incorporating a switch mechanism, the encoding allows for vast flexibility, ensuring every data point is uniquely and efficiently represented.

---

<sup>3</sup>Documentation can be found in the file `README.md`

## 6 Security Analysis, Limitations, and Improvements

While the project perfectly fits the educational requirements of our Python Programming course, it is important to state clearly that this is an academic proof-of-concept. As a genuine cryptographic tool, it has several critical vulnerabilities that we acknowledge and document below.

### 6.1 Lack of Code Obfuscation

Imagine you were a foreign spy in the context of real-world espionage, the source code of a communication tool must be strictly obfuscated. If a counter-intelligence agent intercepts this code, the variable names alone (`encode`, `decode`, `encoded_message`, `cipher`, ...) immediately reveal the exact purpose of the program. If this tool were developed for true covert operations, all variables and function names would be disguised as benign system processes<sup>4</sup> to avoid suspicion.

### 6.2 Pseudorandom Number Generation (PRNG)

A significant cryptographic flaw in our implementation is the use of Python's standard `random` library. The `random` module utilizes an algorithm, which is highly deterministic. If an attacker observes enough outputs from the generator, they can recreate the internal state and predict future index selections.

For a secure implementation, it is absolutely essential to use a Cryptographically Secure Pseudorandom Number Generator (CSPRNG). Implementing `os.urandom()` or Python's built-in `secrets` module would have been significantly safer<sup>5</sup> and would successfully mitigate state-prediction attacks.

### 6.3 Vulnerability of the Switch

As mentioned in section 3.2. The switch acts as a basic Caesar cipher layered on top of the book cipher. Because it is simply a linear addition, it is highly susceptible to brute-force attacks. If the attacker guesses the book correctly, checking for common English words iteratively across a range of possible switch offsets (e.g., 1 . . . 1000) is computationally trivial.

---

<sup>4</sup>For example, `update_metrics`, `log_cache`, ...

<sup>5</sup>Enhanced security measures may result in reduced performance speeds.

## 7 Conclusion

The Secret Message Project was an excellent exercise in applied Python programming.

The Secret Message Project successfully achieved its objective of modernizing the historical Book Cipher into a functional digital tool written in Python. By using Python's dynamic data structures, the project demonstrated how ancient cryptographic logic can be optimized for modern computing. The implementation introduced several critical technical features such as the inclusion of a Switch mechanism added a secondary layer of encryption, while the "Letter Mode" fallback system resolved the traditional limitation of encoding words absent from the source text. We also implemented computational efficiency through a dual-dictionary architecture (`D_words` and `D_letters`), the encoding process was optimized to  $O(n)$  time complexity. The project served as a valuable academic proof-of-concept, allowing for a detailed security analysis of standard Python libraries. We are proud of the work realized and the technical depth achieved. This project stands as a robust educational milestone, bridging the gap between historical cryptographic techniques and modern software engineering principles.



**Cracow University  
of Technology**

## References

- [1] Wikipedia contributors, "Rebecca (novel)", *Wikipedia, The Free Encyclopedia*.  
Available at: [https://en.wikipedia.org/wiki/Rebecca\\_\(novel\)](https://en.wikipedia.org/wiki/Rebecca_(novel))
- [2] Sweigart, A., *Real-World Python: A Hacker's Guide to Solving Problems with Code*  
Available at: <https://nostarch.com/real-world-python/>
- [3] National Library of Ireland, "The 1916 Rising: Personalities and Perspectives."  
Available at: <https://www.nli.ie>
- [4] Project Gutenberg, "The Lost World by Arthur Conan Doyle."  
Available at: <https://www.gutenberg.org/ebooks/139>
- [5] Python Software Foundation, "Python Documentation: random library."  
Available at: <https://docs.python.org/3/library/random.html>
- [6] GitHub, Inc., "Project Repository and Version Control."  
Available at: <https://github.com/>

